

A Quick Tour of Python

Dr. Sumit Kumar Tetarave

ML Lab #1 Lab_1: Part-1

Types in Python

- **int**
 - Bounded integers, e.g. 732 or -5
- **float**
 - Real numbers, e.g. 3.14 or 2.0
- **long**
 - Long integers with unlimited precision
- **str**
 - Strings, e.g. 'hello' or 'C'

Type Conversion (Type Cast)

- Conversion of value of one type to other
- We are used to **int** ↔ **float** conversion in Math
 - Integer 3 is treated as float 3.0 when a real number is expected
 - Float 3.6 is truncated as 3, or rounded off as 4 for integer contexts
- Type names are used as type converter functions

Type Conversion Examples

```
In [20]: int(2.5)  
Out[20]: 2
```

```
In [21]: int(2.3)  
Out[21]: 2
```

```
In [22]: int(3.9)  
Out[22]: 3
```

```
In [23]: float(3)  
Out[23]: 3.0
```

```
In [24]: int('73')  
Out[24]: 73
```

```
In [25]: int('Acads')  
Traceback (most recent call last):
```

```
File "<ipython-input-25-90ec37205222>", line 1, in <module>  
    int('Acads')
```

```
ValueError: invalid literal for int() with base 10: 'Acads'
```

Note that float to int conversion is truncation, not rounding off

```
In [26]: str(3.14)  
Out[26]: '3.14'
```

```
In [27]: str(26000)  
Out[27]: '26000'
```

Type Conversion and Input

```
In [11]: age = input('How old are you? ')
```

```
How old are you? 35
```

```
In [12]: print ('In 5 years, your age will be', age + 5)
```

```
Traceback (most recent call last):
```

```
File "<ipython-input-12-7fb7a9e926c2>", line 1, in <module>
    print ('In 5 years, your age will be', age + 5)
```

```
TypeError: Can't convert 'int' object to str implicitly
```

```
In [13]: print ('In 5 years, your age will be', int(age) + 5)
```

```
In 5 years, your age will be 40
```

Operators

- Arithmetic
- Comparison
- Assignment
- Logical
- Bitwise
- Membership
- Identity

+	-	*	//	/	%	**
---	---	---	----	---	---	----

==	!=	>	<	>=	<=
----	----	---	---	----	----

=	+=	-=	*=	//=	/=	%=	**=
---	----	----	----	-----	----	----	-----

and	or	not
-----	----	-----

&		^	~	>>	<<
---	--	---	---	----	----

in	not in
----	--------

is	is not
----	--------

Assignment Statement

- A simple assignment statement

Variable = Expression;

- Computes the value (object) of the expression on the right hand side expression (**RHS**)
- Associates the name (variable) on the left hand side (**LHS**) with the RHS value
- **=** is known as the assignment operator.

Multiple Assignments

- Python allows multiple assignments

`x, y = 10, 20`

Binds x to 10 and y to 20

- Evaluation of multiple assignment statement:
 - All the expressions on the RHS of the `=` are first evaluated **before any binding happens**.
 - Values of the expressions are bound to the corresponding variable on the LHS.

`x, y = 10, 20`

`x, y = y+1, x+1`

x is bound to 21
and y to 11 at the
end of the program

Binary Operations

Op	Meaning	Example	Remarks
+	Addition	9+2 is 11	
		9.1+2.0 is 11.1	
-	Subtraction	9-2 is 7	
		9.1-2.0 is 7.1	
*	Multiplication	9*2 is 18	
		9.1*2.0 is 18.2	
/	Division	9/2 is 4.25	In Python3
		9.1/2.0 is 4.55	Real div.
//	Integer Division	9//2 is 4	
%	Remainder	9%2 is 1	

The // operator

- Also referred to as “integer division”
- Result is a whole integer (floor of real division)
 - But the type need not be **int**
 - the integral part of the real division
 - rounded towards minus infinity ($-\infty$)
- Examples

9//4 is 2	(-1)//2 is -1	(-1)//(-2) is 0
1//2 is 0	1//(-2) is -1	9//4.5 is 2.0

The % operator

- The remainder operator % returns the remainder of the result of dividing its first operand by its second.

9%4 is 1	(-1)%2 is 1	(-1)//(-2) is 0
9%4.5 is 0.0	1%(-2) is 1	1%0.6 is 0.4

Ideally: $x == (x//y)*y + x \%y$

e.g., $3.14\%0.7$ equals 0.34 (since 3.14 equals $4*0.7 + 0.34$.)

- The modulo operator always yields a result with the same sign as its second operand (or zero);
- the absolute value of the result is strictly smaller than the absolute value of the second operand

Conditional Statements: if-else statement

- Compare two integers and print the min.

```
if x < y:  
    print (x)  
else:  
    print (y)  
print ('is the minimum')
```

1. Check if x is less than y.
2. If so, print x
3. Otherwise, print y.

Indentation

- Indentation is **important** in Python
 - grouping of statement (block of statements)
 - no explicit brackets, e.g. { }, to group statements

➡ `x,y = 6,10`

➡ `if x < y:`

➡ `print (x)`

➡ `else:`

`print (y)`

`print ('The min')`

skipped

Run the program

6

10

Output

6

13

Elif

- A special kind of nesting is the chain of if-else-if-else-... statements
- Can be written elegantly using if-elif-..-else

```
if cond1:
    s1
else:
    if cond2:
        s2
    else:
        if cond3:
            s3
        else:
            ...
```

```
if cond1:
    s1
elif cond2:
    s2
elif cond3:
    s3
elif ...
else
    last-block-of-stmt
```

Evaluation: An example

- Do not evaluate the second operand of binary short-circuit logical operator if the result can be deduced from the first operand
 - Also applies to nested logical operators

`not( (2>5) and (3/0 > 1) ) or (4/0 < 2)`

Evaluates to true

- ✓ Python evaluates expressions from left to right
- ✓ While evaluating an assignment, the right-hand side is evaluated before the left-hand side.

If-else: An example

- What is the output of the following program:

```
y = 0.1*3  
if y != 0.3:  
    print ('Launch a Missile')  
else:  
    print ("Let's have peace")
```

Launch a Missile

Caution about Using Floats

- Representation of *real numbers* in a computer can not be exact
 - Computers have limited memory to store data
 - *Between any two distinct real numbers, there are infinitely many real numbers.*
- On a typical machine running Python, there are 53 bits of precision available for a Python float

Caution about Using Floats

- The value stored internally for the decimal number 0.1 is the binary fraction

0.00011001100110011001100110011001100110011001100110011010

- Equivalent to decimal value

0.10000000000000000055511151231257827021181583404541015625

- Approximation is similar to decimal approximation $1/3 = 0.3333333333...$
- No matter how many digits you use, you have an approximation

Comparing Floats

- Because of the approximations, comparison of floats is not exact.
- **Solution?**
- Instead of

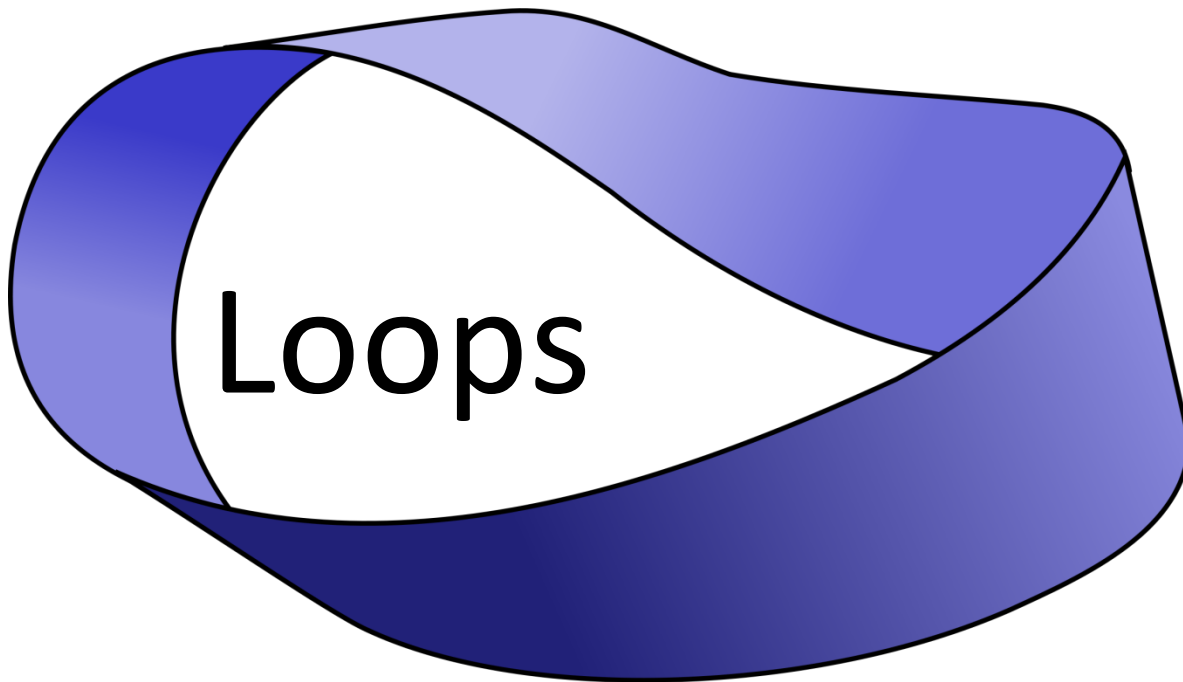
$x == y$

use

$\text{abs}(x-y) \leq \text{epsilon}$

where **epsilon** is a suitably chosen small value

Programming using Python

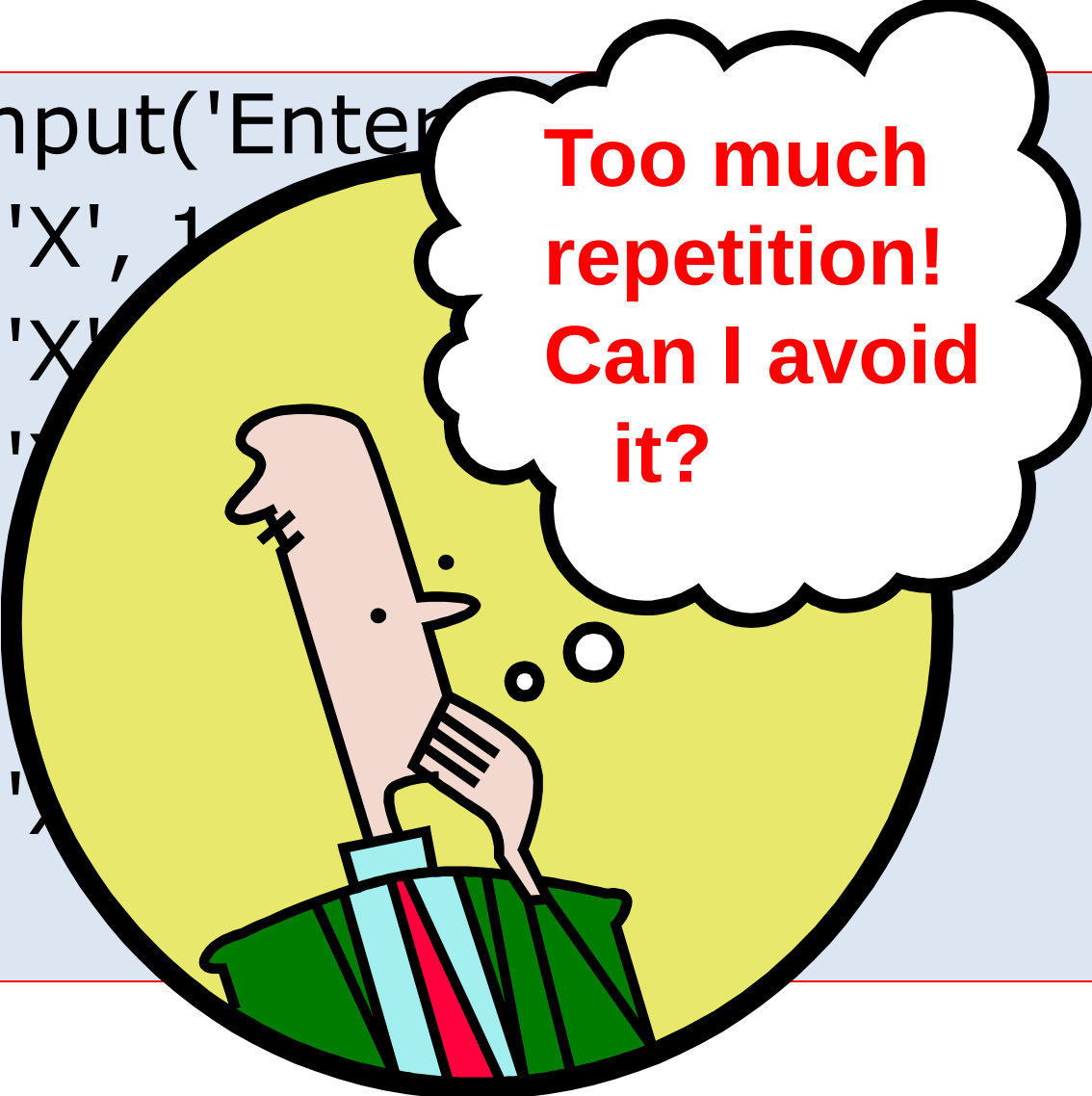


Printing Multiplication Table

5	X	1	=	5
5	X	2	=	10
5	X	3	=	15
5	X	4	=	20
5	X	5	=	25
5	X	6	=	30
5	X	7	=	35
5	X	8	=	40
5	X	9	=	45
5	X	10	=	50

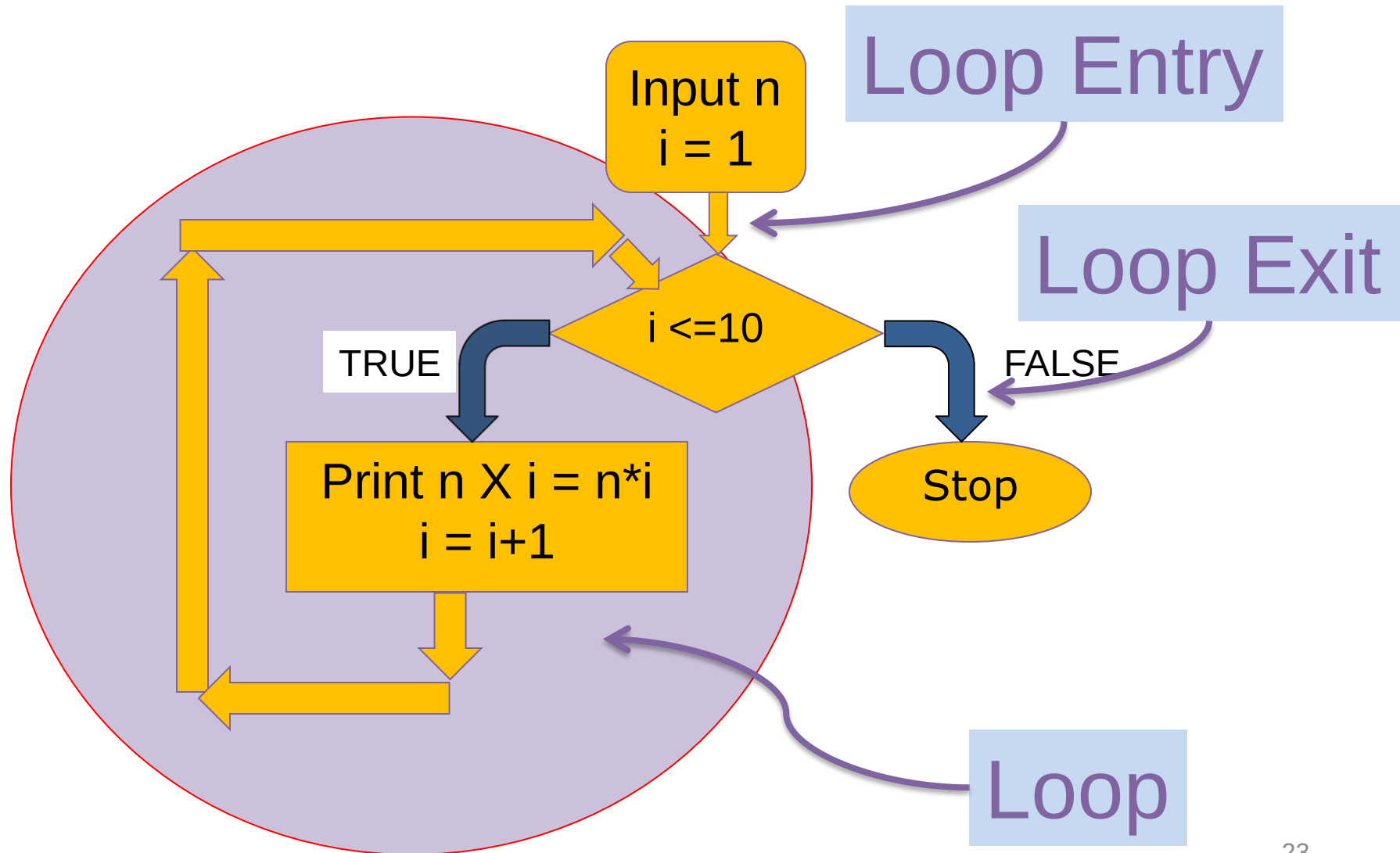
Program...

```
n = int(input('Enter a number: '))  
print (n, 'X', 1)  
print (n, 'X', 2)  
print (n, 'X', 3)  
print (n, 'X', 4)  
print (n, 'X', 5)  
print (n, 'X', 6)  
print (n, 'X', 7)  
print (n, 'X', 8)  
print (n, 'X', 9)  
print (n, 'X', 10)  
....
```

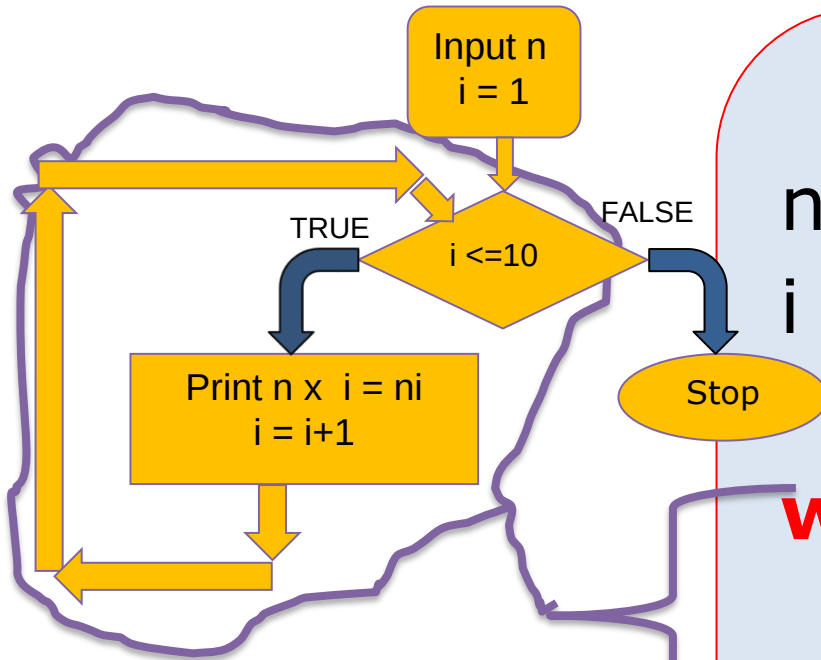


**Too much
repetition!
Can I avoid
it?**

Printing Multiplication Table



Printing Multiplication Table



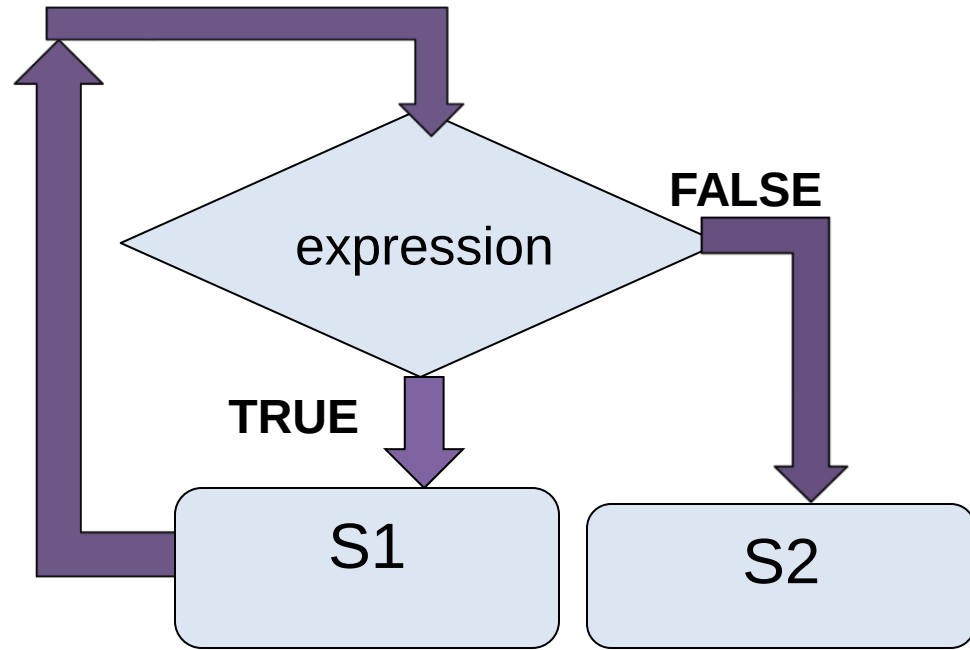
```
n = int(input('n=? '))
```

```
i = 1
```

```
while (i <= 10) :  
    print (n , 'X', i, '=', n*i)  
    i = i + 1  
print ('done')
```

While Statement

```
while (expression):  
    S1  
S2
```



1. Evaluate expression
2. If TRUE then
 - a) execute statement1
 - b) goto step **1**.
3. If FALSE then execute statement2.

For Loop

- Print the sum of the reciprocals of the first 100 natural numbers.

```
rsum=0.0# the reciprocal sum

# the for loop
for i in range(1,101):
    rsum = rsum + 1.0/i
print ('sum is', rsum)
```

For loop in Python

- General form

```
for variable in sequence:  
    stmt
```

range

- `range(s, e, d)`
 - generates the list:
 $[s, s+d, s+2*d, \dots, s+k*d]$
where $s+k*d < e \leq s+(k+1)*d$
- `range(s, e)` is equivalent to `range(s, e, 1)`
- `range(e)` is equivalent to `range(0, e)`

Exercise: What if `d` is negative? Use python interpreter to find out.

Example

- What will be the output of the following program

```
# print all odd numbers < 10
i = 1
while i <= 10:
    if i%2==0: # even
        continue
    print (i, end= ' ')
    i = i+1
```

Continue and Update Expr

- Make sure continue does not bypass update-expression for while loops



```
# print all odd numbers < 10
```

```
i = 1
```

```
while i <= 10:
```

```
    if i%2==0: # even
```

```
        continue
```

```
    print (i, end=' ')
```

```
    i = i+1
```

i is not incremented
when even number
encountered.
Infinite loop!!

```
i = 1
while i <= 10:
    i = i+1
    if i%2==0: # even
        #print (i)
        continue
    print (i, end=' ')
    #i = i+1
```

3 5 7 9 11

To check "end" parameter
for i in range(5):
 print(i, end=' ')

the end=' ' parameter ensures
that each value of i is printed
on the same line, separated
by a space.